

KV-aware LLM serving stack

From application-visible reuse hints to model execution, memory tiers, and transfer paths.

LAYER LEGEND

L1 App/workflow

L2 Control

L3 Metadata

L4 Reuse

L5 KV service

L6 Engine

L7 Model + memory/transport

Right cards show where research/tools attach to the stack.

CONCEPTUAL LAYER	INTERFACE CONTROLLED	REPRESENTATIVE SYSTEMS AND PAPERS
L1 Application / Agent runtime stable segments, workflow graph, tool stalls, privacy labels	Declares what is cacheable and what may be shared.	Prompt CacheKVFlowTokencakeAgentix
L2 KV-aware router / scheduler cache affinity, load balance, SLO, admission	Chooses route-to-state, move-state-to-request, or recompute.	MooncakeDualMapJITServeIIm-d
L3 Compatibility / metadata plane model id, tokenizer, RoPE, dtype, tenant, TTL	Defines whether one KV object is safe and useful for another request.	SCXKV ABICache indexesprovenance
L4 Reuse and correction engine prefix hit, chunk reuse, semantic match, repair	Turns stored state into usable computation savings.	LMCacheKVShareSemShareKVDroidSpeak
L5 External KV state service tiered store, global cache, transfer orchestration	Materializes KV outside the local engine and across service boundaries.	CacheGenHCacheMooncake StoreNIXL
L6 Inference engines paged KV allocation, radix cache, kernels, workers	Owns live execution and local KV layout.	vLLMSGLangTensorRT-LLMDynamo
L7 Model + Memory / Transport attention shape, KV pages, HBM/DRAM/SSD/remote tiers, RDMA/NVLink/NVMe-oF	Determines KV footprint, placement cost, and movement path. This is where model architecture, storage tiering, and network transport meet.	attention shapeKV pagesHBM/DRAM/SSDremote KVRDMA/NIXLNVMe-oFcompressionretention

HBM hot

CPU DRAM warm

SSD/NVMe cold

Remote KV service

RDMA/NIXL transport

Local runtime and exact reuse

L6L7L4

vLLMPagedAttentionSGLang

Prompt Cache

KV becomes allocatable local state and structured prefix reuse.

Substrate: strongest when prefixes or modules align.

Phase-disaggregated serving

L2L5L7

SplitwiseDistServeMooncake

PrfaaS

KV becomes the prefill-to-decode handoff object.

Boundary: execution architecture depends on transfer.

Movement, tiering, restoration

L3L5L7

CacheGenInfiniGenHCache

IceCacheLouisKViHiFC

Controls what moves, what restores, and which tier is used.

Risk: transfer latency or wrong retrieval kills savings.

Distributed KV serving and control

L2L3L5L6

LMCacheMooncake TEDynamo/NIXL

IIm-dDualMap

Promotes KV into cluster control state for route, transfer, and SLOs.

Control plane: locality versus load balancing.

Advanced reuse and workflow

L1L3L4

KVShareSemShareKVDroidSpeak

KVCOMMQ-KVCommTokenDance

Extends reuse beyond exact prefix into semantic, cross-model, and agentic settings.

Compatibility contract becomes the main systems interface.

Governance and workload realism

L1L2L3L5

SCXKVCache in the WildServeGen

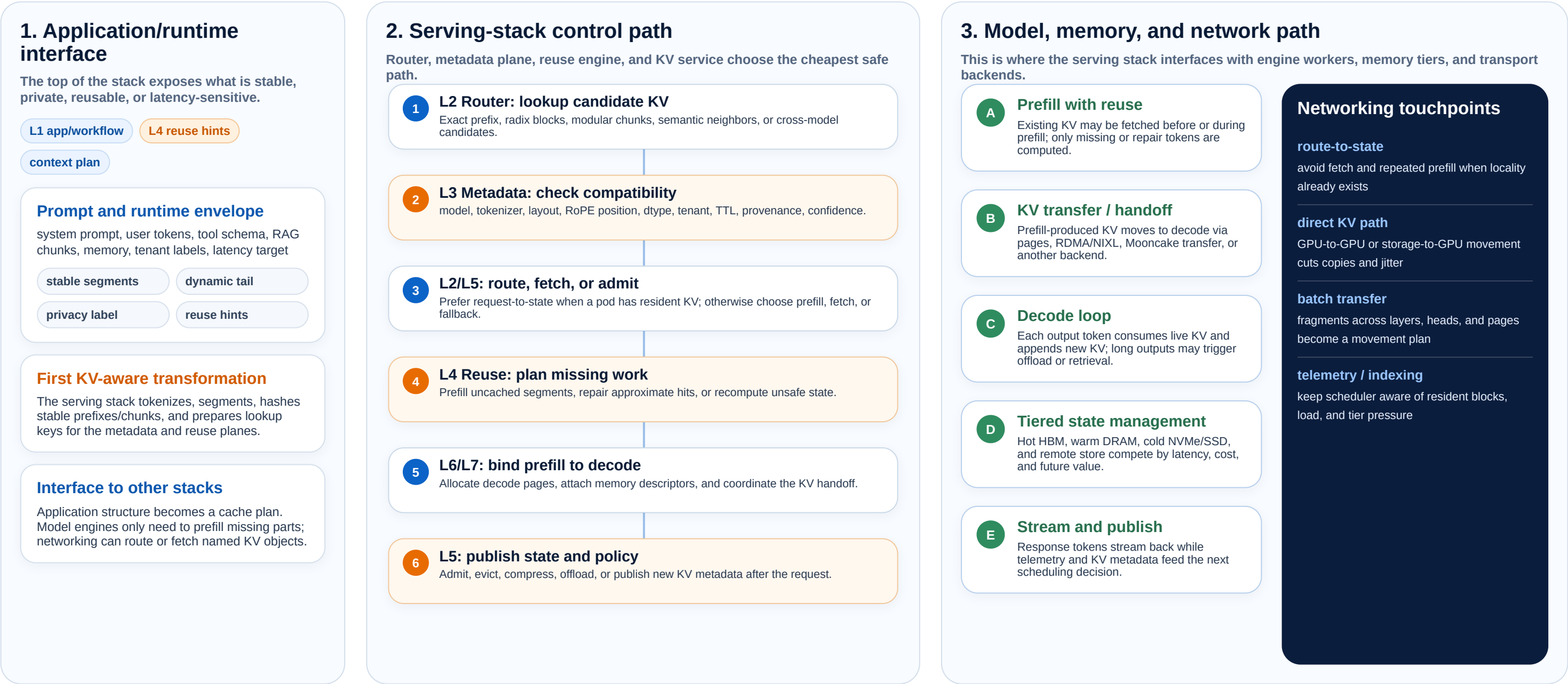
tenant isolation

Externalized KV needs provenance, tenant controls, traces, and safe fallback.

Guardrails decide whether reuse is allowed.

Serving stack in action: from API request to KV-aware execution

Page 1's stack turns a prompt into control-plane decisions, compatibility checks, reusable KV state, and model/memory/transport execution.



Networking's role in the token economy

The network adds value when KV changes from anonymous tensor bytes into addressable state.

REUSE VALUE PROXY

$$\text{net_value} \sim \text{p_hit} * \text{C_saved}$$

- C_store

- p_fetch * C_transfer

- C_fallback

- C_control

- C_quality

p_hit +

Context engineering raises reuse probability

Stable system prompts, tool schemas, documents, plans, and memory summaries expose high-value cache boundaries.

C_transfer -

Serving software lowers movement cost

External KV layers, tiered stores, transfer engines, and cache-aware routers turn KV into a schedulable object.

Minimum KV object metadata

model

tokenizer

layer layout

block size

RoPE offset

dtype

compression

tenant

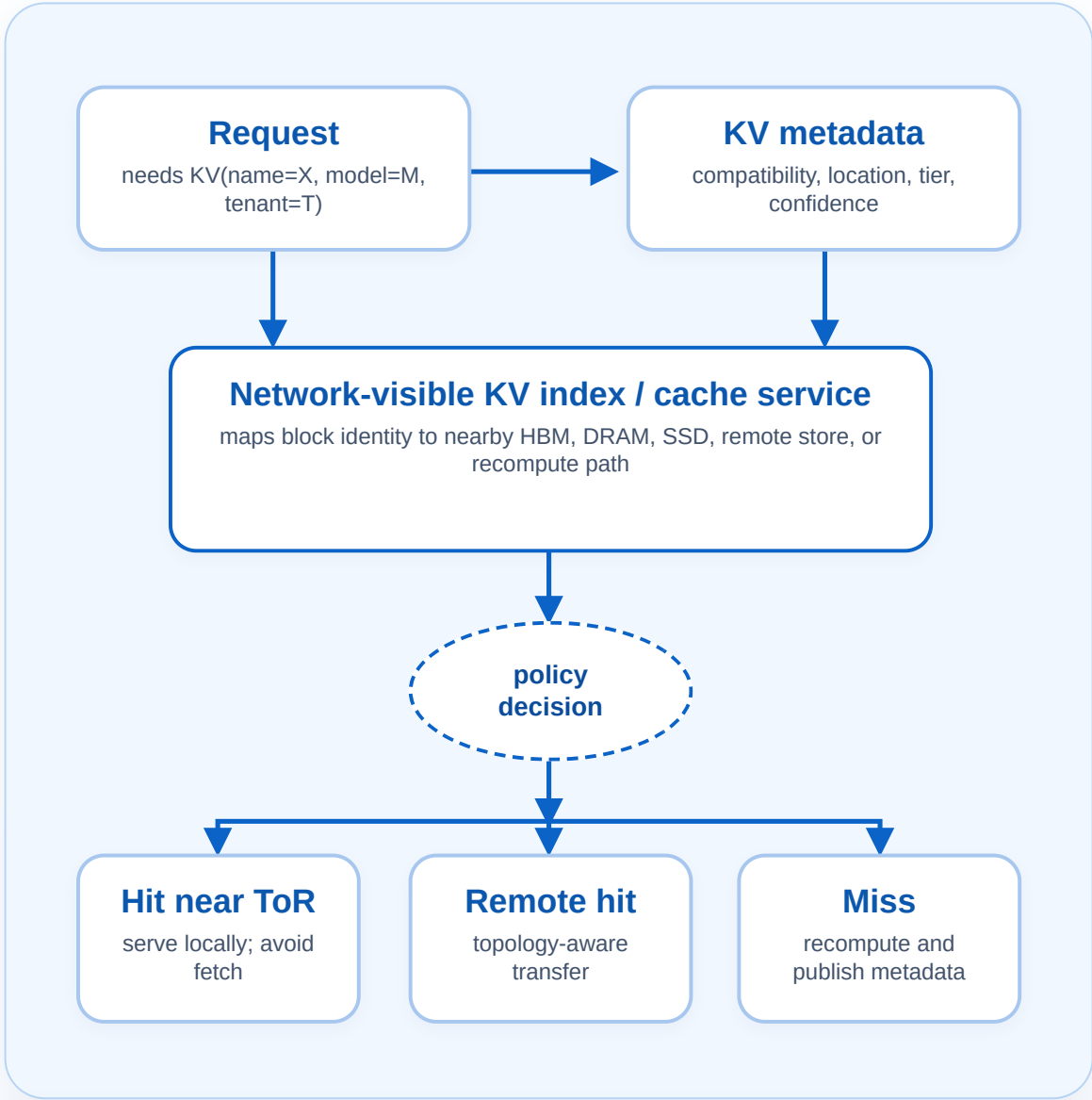
TTL

provenance

confidence

policy

Route request to KV, or move KV to request?



- 1

Route-to-state
Best transfer is no transfer. Scheduler scores pods by resident prefix or block fraction and routes toward locality.
- 2

Direct memory path
Prefill writes KV directly into decode GPU memory through RDMA, NIXL, or Mooncake-style transport.
- 3

Batch / scatter-gather transfer
Fragmented KV pages become one transfer plan across VRAM, DRAM, NVMe-oF, and remote tiers.
- 4

Storage-to-GPU bypass
Restore cold KV without CPU bounce buffers when direct DMA or GDS-like paths are available.
- 5

Request-driven placement
Name, tenant policy, reuse confidence, and telemetry let edge or ToR-attached services admit, fetch, or reject KV.

Bottom line

KV reuse saves compute; networking preserves the savings by avoiding unnecessary fetches, avoiding CPU copies, batching fragmented movement, and making KV blocks routable by identity rather than only movable as byte streams.